★ COMMON INTERVIEW QUESTION ★

Chunking Strategy

Interview Guide

Strategy → Best Document Type → Trade-offs

A Complete Reference for LLM / RAG Engineer Interviews

Created: June 26, 2026 | 22:29

LangChain • RAG • NLP • Vector Databases • LLM Engineering

Table of Contents

1. Why Interviewers Ask About Chunking

Chunking strategy is one of the most frequently asked RAG / LLM engineering interview questions because it sits at the intersection of **information retrieval, NLP, and system design**. Getting it wrong is the #1 cause of poor RAG answer quality in production systems.

Interviewers use this question to test whether you can:

- Distinguish between at least 4–5 distinct chunking strategies by name
- Map each strategy to the document types it is best suited for
- Articulate concrete trade-offs (not just 'it depends')
- Demonstrate awareness of LangChain / production tooling
- Think end-to-end: how does chunk quality affect retrieval scores and LLM answers?

> ★ **Interview Key Point** A strong answer covers ≥ 4 strategies, names at least one LangChain class per strategy, and ties chunk quality back to retrieval accuracy and final answer quality.

2. The Master Summary Table

This is the core reference. Each row is a strategy; columns map to document type, pros, cons, when to avoid, and LangChain class. Memorise one row per strategy for interviews.

Strategy	How It Works	Best Document Type	Pros	Cons	When to Avoid	LangChain Class
Fixed-Size (Character)	Split every N chars; repeat last M chars (overlap)	Plain text, newsletter, generic prose	Zero cost Simple Fast Deterministic	Cuts mid-sentence No semantic awareness Context loss	Multi-topic docs, code, structured markdown	RecursiveCharacterTextSplitter (chunk_size, chunk_overlap)
Fixed-Size (Token)	Split every N tokens using model tokeniser (e.g. tiktoken)	Any text when token budget is critical constraint	Precise token budget control Model-aligned No truncation risk	More complex setup Slightly slower than char split	When char count is sufficient; open-source models	RecursiveCharacterTextSplitter (length_function= tiktoken)
Paragraph (Regex \n\n)	Split on blank lines (double newline); filter by size	Blog posts, news articles, academic abstracts, books	Preserves full paragraph context Free (no embedding) Clean boundaries	Paragraph length varies wildly No guarantee on chunk size	Code files, markdown with headings, tables, CSV/JSON	RecursiveCharacterTextSplitter separators= ['\n\n','\n',' ','']
Section / Heading (Regex or splitter)	Split at heading markers (#, ##) or numbered sections; preserve heading	Markdown docs, Wikipedia, legal contracts, tech manuals	Preserves section structure & title Natural boundaries Metadata-rich	Only works with well-structured docs Sections can be very long	Plain prose with no headings; PDF scans without clear structure	MarkdownHeaderTextSplitter HTMLHeaderTextSplitter
Sentence (NLP tokeniser)	Use spacy/nltk to detect sentence ends; groupsentences up to N chars	Academic papers, legal clauses, fine-grained Q&A, chatbot FAQ	Cleanest semantic boundaries No mid-sentence cuts Great for dense text	Requires NLP library Many small chunks = higher embed cost	Code, tables, bullet lists; docs with very long sentences	NLTKTextSplitter SpacyTextSplitter (sentence-level)
Semantic (Embedding-base	Embed each sentence; place boundary where cosine similarity drops < threshold	Mixed-topic essays, research papers, long blog posts, transcripts	Truly topic-aware Best retrieval quality No fixed size limit	Needs embedding call per sentence Higher cost & latency Non-deterministic	Short docs where fixed-size is fine; real-time indexing (too slow)	SemanticChunker (langchain-experimental)

Strategy	How It Works	Best Document Type	Pros	Cons	When to Avoid	LangChain Class
Parent-Child (Hierarchical)	Index small child chunks; store large parent; retrieve child, return parent	Medical records, legal documents, tech handbooks, RAG production	Best precision AND best context No context loss Highly accurate	Complex setup Two chunk sizes to tune Higher storage use	Simple prototypes; cases where parent size still dilutes relevance	ParentDocument Retriever (child + parent splitter)
Agentic / Late Chunking	LLM reads doc and decides chunk boundaries based on meaning	Complex reports, mixed-format docs, research requiring deep understanding	Highest quality chunk boundaries Handles irregular structure	Expensive (LLM call per doc) Very slow Not scalable	High-volume indexing pipelines; when cost matters; large corpora	Custom LLM chain (no standard LangChain class yet)

Table 1 — Master chunking strategy reference (8 strategies × 6 dimensions)

3. Quick One-Line Reference

Use these one-liners when the interviewer asks for a rapid overview.

Strategy	One-Line Summary
Fixed-Char	Split every N characters — fastest, simplest, best default starting point.
Fixed-Token	Split every N tokens — use when your model's token limit must be respected precisely.
Paragraph	Split on blank lines — ideal for prose where paragraphs are the natural unit of thought.
Section	Split on headings — ideal for structured markdown/HTML/legal docs with clear hierarchy.
Sentence	Split on sentence boundaries — best for fine-grained Q&A and dense academic text.
Semantic	Split where topic shifts (cosine sim drop) — best quality, highest cost.
Parent-Child	Index small chunks, fetch large parent — best of both worlds for production RAG.
Agentic	LLM decides boundaries — highest quality, not scalable; use only for critical small corpora.

Table 2 — One-sentence summary per strategy for rapid recall

4. Decision Flowchart — Which Strategy to Pick

When asked 'how would you decide which chunking strategy to use?', walk the interviewer through this decision tree. It demonstrates structured, systematic thinking.

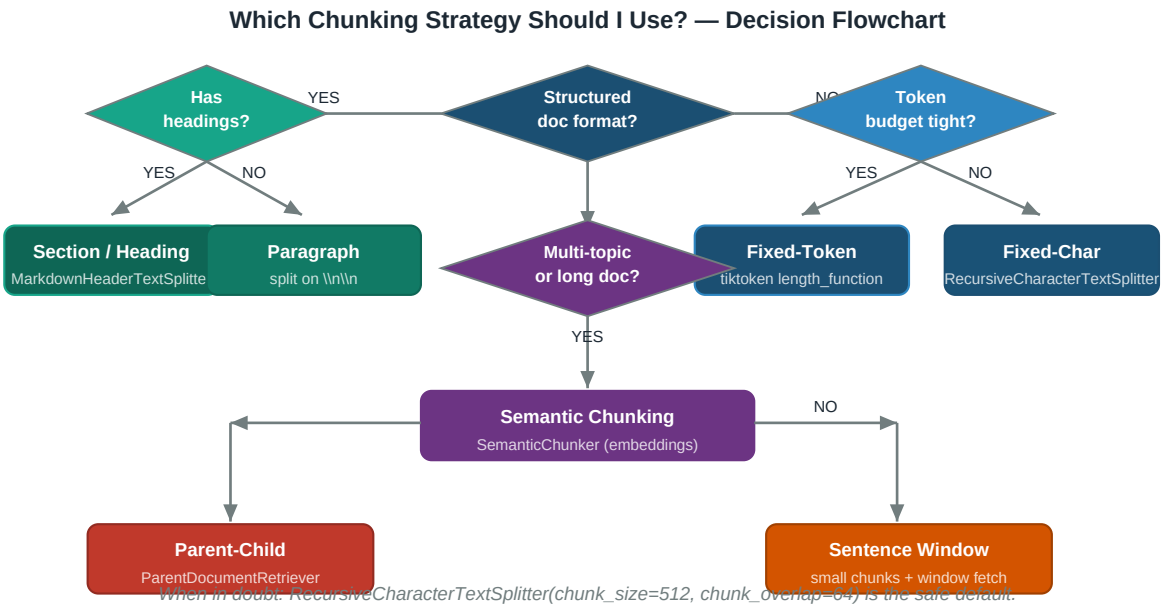

Figure 1 — Decision flowchart for selecting a chunking strategy

5. Strategy Comparison Radar

The chart below scores each strategy 1–5 across five dimensions. Use it to quickly visualise trade-offs when comparing two strategies side by side.

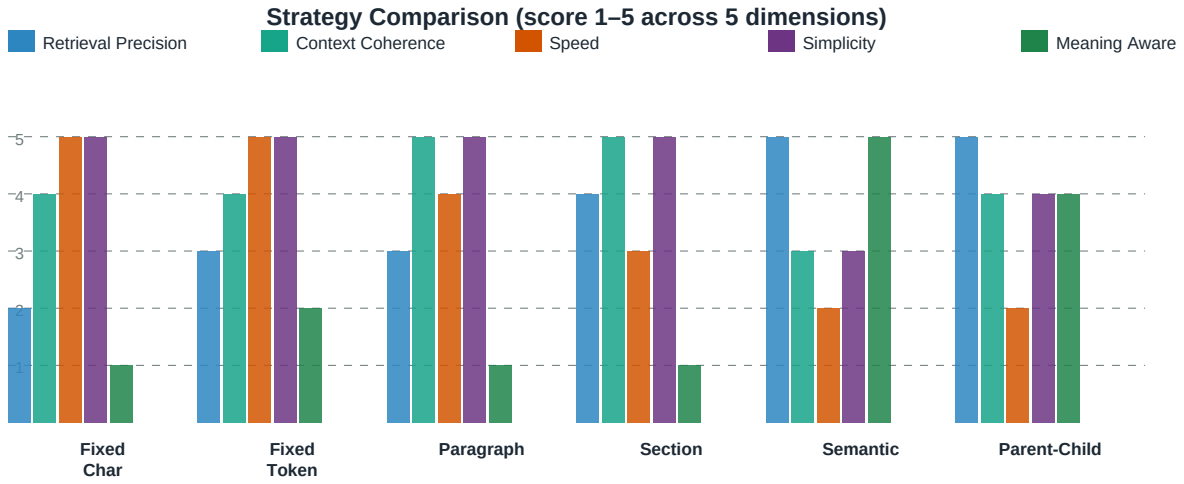

Figure 2 — Strategy scores across Retrieval Precision, Context Coherence, Speed, Simplicity, Meaning-Awareness

■ No single strategy scores 5 on all axes — that is the point of the trade-off. Parent-Child comes closest but requires the most complex setup.

6. Deep Dive: Each Strategy with Code

6.1 Fixed-Size (Character)

The simplest strategy. Split every chunk_size characters, repeat the last chunk_overlap characters at the start of the next chunk.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,      # characters per chunk
    chunk_overlap=64,    # 12.5% overlap — bridges boundary context
    length_function=len,
    add_start_index=True,
)
chunks = splitter.split_text(open("article.txt").read())
# Best for: plain prose, newsletters, generic text
# Avoid for: structured markdown, multi-topic docs, code
```

6.2 Fixed-Size (Token)

Same as character-based but measures size in **tokens**. Essential when your embedding model has a strict token limit (e.g. text-embedding-3-small: 8191 tokens).

```
import tiktoken
from langchain_text_splitters import RecursiveCharacterTextSplitter

enc = tiktoken.get_encoding("cl100k_base")   # GPT-4 / Ada-002 tokeniser

splitter = RecursiveCharacterTextSplitter(
    chunk_size=256,             # 256 TOKENS (not chars)
    chunk_overlap=32,
    length_function=lambda t: len(enc.encode(t)),
)
chunks = splitter.split_text(open("report.txt").read())
# Best for: any text when token budget must be controlled precisely
# Avoid for: when char-count is sufficient; local models without tiktoken
```

6.3 Paragraph (Regex on \\n\\n)

Split on double newlines — the natural paragraph boundary in plain text. Zero embedding cost; pure regex.

```
import re

def split_paragraphs(text: str, min_len=80, max_len=2000) -> list[str]:
    chunks = re.split(r'\n\n+', text)
    result = []
    for c in chunks:
        c = c.strip()
        if len(c) < min_len:
            continue
        if len(c) > max_len:
            # secondary split on single newline
            result.extend(p.strip() for p in c.split('\n') if len(p.strip()) >= min_len)
        else:
            result.append(c)
    return result

chunks = split_paragraphs(open("blog_post.txt").read())
# Best for: blog posts, news articles, academic abstracts, books
# Avoid for: structured markdown with headings, code, CSV/JSON
```

6.4 Section / Heading (MarkdownHeaderTextSplitter)

Splits at heading markers and attaches the heading text as metadata. Retrieval can then filter or weight by heading level.

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

# Define which heading levels to split on
headers_to_split = [
    ("#",   "h1"),
    ("##",  "h2"),
    ("###", "h3"),
]

splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split,
    strip_headers=False,   # keep heading in chunk content
)

with open("handbook.md") as f:
    md_text = f.read()

docs = splitter.split_text(md_text)
for doc in docs:
    # doc.metadata = {"h1": "Introduction", "h2": "Background", ...}
    print(f"[{doc.metadata}]  {doc.page_content[:80]}...")

# Best for: Markdown, Wikipedia, legal contracts, technical manuals
# Avoid for: plain prose without headings, PDF scans
```

6.5 Sentence (NLTKTextSplitter / SpacyTextSplitter)

Uses a proper NLP tokeniser to detect sentence endings — handles abbreviations like 'Dr.', 'U.S.A.' and ellipses that trip up simple regex.

```
from langchain_text_splitters import NLTKTextSplitter
import nltk
nltk.download("punkt", quiet=True)

splitter = NLTKTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
)
chunks = splitter.split_text(open("research_paper.txt").read())

# OR with spaCy (better for multi-language / complex text):
from langchain_text_splitters import SpacyTextSplitter

spacy_splitter = SpacyTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    pipeline="en_core_web_sm",   # pip install spacy; python -m spacy download en_core_web_sm
)
chunks = spacy_splitter.split_text(open("research_paper.txt").read())
# Best for: academic papers, legal clauses, dense Q&A
# Avoid for: code, bullet lists, tables
```

6.6 Semantic (SemanticChunker)

Embeds every sentence and places chunk boundaries where cosine similarity between consecutive sentences drops sharply — a true topic-shift detector.

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings
# OR free local option:
from langchain_huggingface import HuggingFaceEmbeddings

# Option A: OpenAI (better quality, paid)
emb = OpenAIEmbeddings(model="text-embedding-3-small")

# Option B: local (free)
# emb = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

chunker = SemanticChunker(
    embeddings=emb,
    breakpoint_threshold_type="percentile",   # recommended
    breakpoint_threshold_amount=95,           # top 5% similarity drops = boundaries
)
chunks = chunker.split_text(open("mixed_topic_essay.txt").read())
# Best for: mixed-topic essays, research papers, transcripts
# Avoid for: real-time indexing (slow); short docs where fixed-size is fine
```

6.7 Parent-Child (ParentDocumentRetriever)

The production-grade solution: index tiny child chunks for retrieval precision, but return the full parent chunk to the LLM for rich context.

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import TextLoader

docs = TextLoader("medical_handbook.txt").load()

child_splitter  = RecursiveCharacterTextSplitter(chunk_size=200,  chunk_overlap=20)
parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1500, chunk_overlap=100)

vectorstore = FAISS.from_documents([], OpenAIEmbeddings())
docstore    = InMemoryStore()

retriever = ParentDocumentRetriever(
    vectorstore=vectorstore,
    docstore=docstore,
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
retriever.add_documents(docs)   # indexes children, stores parents

# At query time: finds best child, returns parent to LLM
results = retriever.invoke("treatment protocol for Type 2 diabetes")
# Best for: production RAG, medical/legal/technical docs
# Avoid for: simple prototypes; when storage is constrained
```

7. Sample Interview Q&A;

These are real interview questions from FAANG, AI startups, and ML engineering roles. Model answers are structured: name → doc type → pro → con → LangChain class.

Q: What chunking strategies do you know, and how would you choose between them?

A: There are roughly 6–8 strategies. For plain prose I use paragraph splitting (split on \n\n) — free, fast, respects natural boundaries. For structured docs like Markdown I use MarkdownHeaderTextSplitter to preserve section hierarchy as metadata. For multi-topic or long documents I use SemanticChunker from langchain-experimental — it embeds each sentence and places boundaries where cosine similarity drops, making chunks genuinely topic-aware. For production RAG where I need both retrieval precision AND full LLM context, I use ParentDocumentRetriever — index small child chunks, return large parent chunks. My default starting point for any new project is RecursiveCharacterTextSplitter with chunk_size=512 and chunk_overlap=64, then I tune based on retrieval quality.

Q: What are the trade-offs between small and large chunk sizes?

A: Small chunks give high retrieval precision but lose surrounding context — the retrieved chunk may contain the answer without the context needed to interpret it. Large chunks preserve context but dilute the embedding signal across multiple topics, reducing similarity scores for any specific query. The fix: use chunk_overlap (10–20% of chunk_size) to bridge boundary context, or use ParentDocumentRetriever to get precise retrieval with full context delivery.

Q: When would you NOT use semantic chunking despite its quality advantages?

A: Three cases: (1) Real-time or high-volume indexing pipelines — semantic chunking requires one embedding per sentence, which is ~10× slower and more expensive than character splitting. (2) Short, uniform documents where topic shifts don't occur — a single-topic 500-word article doesn't need semantic splitting; fixed-size works fine. (3) When the embedding model is the same for chunking and retrieval — you'd be optimising for yourself, and a cheaper local model (all-MiniLM-L6-v2) is fine for boundary detection.

Q: How does chunking strategy affect RAG answer quality?

A: Chunking is the #1 factor in RAG quality — more than the choice of LLM. If retrieved chunks don't contain the answer or are full of irrelevant content, even GPT-4 cannot produce a correct answer. The chain is: chunk quality → embedding precision → retrieval rank → LLM context quality → answer. Poor chunking manifests as: LLM saying 'I don't know' when the doc has the answer (chunks too small — context stripped), or LLM giving vague answers mixing unrelated topics (chunks too large — diluted embedding).

Q: What is the difference between MarkdownHeaderTextSplitter and RecursiveCharacterTextSplitter?

A: RecursiveCharacterTextSplitter is general-purpose — it uses a priority list of separators (\n\n, \n, space, char) and enforces a hard chunk_size limit. MarkdownHeaderTextSplitter is structure-aware — it splits specifically at heading markers (#, ##, ###) and attaches heading text as chunk metadata (e.g. {'h1': 'Introduction', 'h2': 'Methods'}). The key difference: MHTS doesn't enforce a size limit, so a long section becomes one large chunk. Best practice: use MHTS first for structure, then feed oversized chunks to RecursiveCharacterTextSplitter as a secondary size guard.

8. Common Follow-up Questions

After the main chunking question, interviewers typically probe deeper with one of these. Prepare a 2–3 sentence answer for each.

Q: How do you evaluate whether your chunking strategy is working?

- Run 10–20 representative queries against the vector store.
- Inspect the top-3 retrieved chunks manually — do they contain the full answer?
- Measure retrieval metrics: MRR (mean reciprocal rank), Recall@K, NDCG.
- Check end-to-end answer quality with an LLM-as-judge eval pipeline.

Q: What is chunk_overlap and when would you set it to zero?

- chunk_overlap repeats the last N characters of chunk N-1 at the start of chunk N.
- Set it to zero only when chunks are guaranteed to be semantically self-contained (e.g. database rows, FAQ items).
- For prose, always use 10–20% overlap to prevent boundary context loss.

Q: How does chunking interact with embedding model token limits?

- Embedding models have a hard token limit (e.g. text-embedding-3-small: 8191 tokens).
- Chunks exceeding the limit are silently truncated — the tail is dropped from the embedding.
- Use tiktoken as length_function to measure in tokens, not characters, and stay under the limit.

Q: What is contextual retrieval and how does it relate to chunking?

- Contextual retrieval (Anthropic) prepends an AI-generated context summary to each chunk before embedding.
- It solves the 'decontextualised chunk' problem without increasing chunk size.
- The trade-off: one LLM call per chunk during indexing; use a cheap model (Haiku / gpt-4o-mini).

Q: How would you chunk a PDF with mixed text and tables?

- Use an unstructured document loader (Unstructured.io) to extract text and tables separately.
- Chunk text with RecursiveCharacterTextSplitter; represent tables as serialised markdown or JSON.
- Store table chunks with metadata (page_number, table_index) for accurate citation.

9. How Interviewers Score Your Answer

Evaluation Criterion	Weak Answer	Strong Answer
Names the strategy	Just says 'chunking'	Names ≥3 strategies with correct terminology
Links to document type	Generic 'it depends'	Specific: 'Semantic chunking for multi-topic research papers'
States trade-offs	Only mentions pros	Balanced: 1 clear pro + 1 clear con per strategy
Mentions LangChain class	No implementation detail	Cites RecursiveCharacterTextSplitter, SemanticChunker, etc.
Handles follow-ups	Repeats the same answer	Pivots to overlap, token limits, ParentDocumentRetriever
Demonstrates RAG awareness	Talks about chunking in isolation	Ties chunk quality to retrieval score & LLM answer quality

Table 3 — Interviewer evaluation rubric: weak vs. strong answers

■ The most common mistake: answering 'it depends on the use case' without specifying what it depends on. Always follow up with concrete examples: 'For a Wikipedia RAG system I'd use MarkdownHeaderTextSplitter because...'

10. Model Answer Template

Use this 5-step structure to answer any chunking strategy question confidently and completely within 90 seconds.

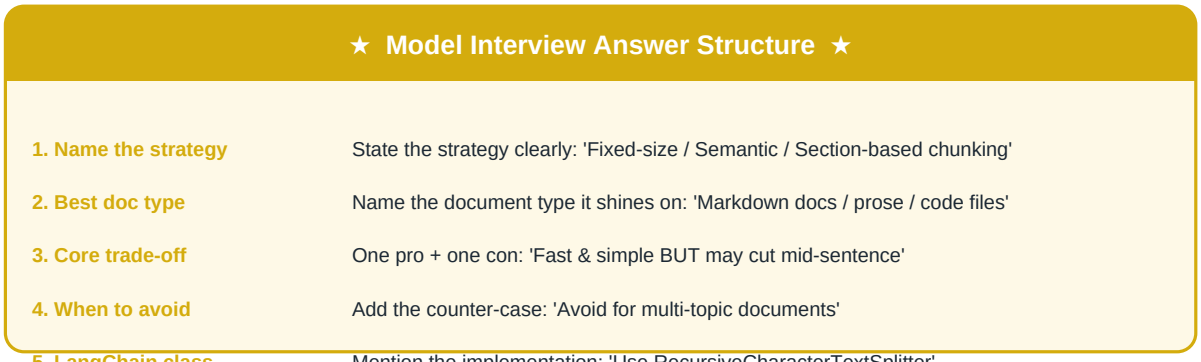

Figure 3 — 5-step answer structure for chunking strategy interview questions

10.1 Example Using the Template

Step	For: Semantic Chunking
1. Name	Semantic or embedding-based chunking
2. Doc type	Mixed-topic essays, research papers, long transcripts
3. Trade-off	PRO: topic-aware boundaries, highest retrieval quality CON: requires one embedding per sentence — high cost & latency
4. Avoid when	Real-time indexing, short uniform docs, cost-sensitive pipelines
5. LangChain	SemanticChunker from langchain-experimental; HuggingFaceEmbeddings or OpenAIEmbeddings as backend

11. References

[1] LangChain Docs — Text Splitters Overview https://python.langchain.com/docs/concepts/text_splitters/

[2] LangChain Docs — RecursiveCharacterTextSplitter https://python.langchain.com/docs/how_to/recursive_text_splitter/

[3] LangChain Docs — MarkdownHeaderTextSplitter
https://python.langchain.com/docs/how_to/markdown_header_metadata_splitter/

[4] LangChain Experimental — SemanticChunker https://python.langchain.com/docs/how_to/semantic-chunker/

[5] LangChain Docs — ParentDocumentRetriever https://python.langchain.com/docs/how_to/parent_document_retriever/

[6] Anthropic — Contextual Retrieval https://www.anthropic.com/news/contextual-retrieval

[7] Pinecone — Chunking Strategies for LLM Applications https://www.pinecone.io/learn/chunking-strategies/

[8] Greg Kamradt — 5 Levels of Chunking https://github.com/FullStackRetrieval-com/RetrievalTutorials

[9] OpenAI — Embeddings & Token Limits https://platform.openai.com/docs/guides/embeddings

[10] tiktoken — OpenAI Tokenizer https://github.com/openai/tiktoken

---

Generated on June 26, 2026 at 22:29. Based on LangChain v0.2+, Python 3.10+.